LLM Quantization Benchmark Report

Model : Mistral 7B Instruct v0.3
Date : 2026-03-30
Methods : FP16 Baseline, BitsAndBytes NF4, AWQ INT4, AWQ INT4 (Marlin), GPTQ INT4 (Marlin), GPTQ INT4 (Generic), FP8 W8A8
GPUs : NVIDIA A100 SXM 80GB, NVIDIA L4 24GB, NVIDIA RTX A5000 24GB
GitHub Link : https://github.com/NitinDatta8/quantization-benchmarking

1. Introduction

Quantization offers a path to reduce inference cost and latency, but introduces tradeoffs in output quality and scaling behavior that must be measured, not assumed.

This study benchmarks seven inference configurations - one unquantized baseline, four quantization methods, and two kernel-isolation variants - on a single 7B-parameter instruction-tuned model, using 65 domain-aligned prompts across five personalization categories. Each method is primarily evaluated on the GPU architecture it is best suited for, with additional cross-hardware runs and kernel-swap experiments to isolate the contribution of inference kernels versus quantization algorithms.

2. Background

Post-training quantization (PTQ) reduces the numerical precision of model weights (and optionally activations) after training, shrinking memory footprint and often improving throughput. The methods tested fall into two categories:

Weight-only quantization compresses weights to lower precision while keeping activations in FP16. This reduces memory bandwidth requirements during inference, which is typically the bottleneck for autoregressive generation. AWQ, GPTQ, and BitsAndBytes NF4 all fall into this category at 4-bit precision, but differ in how they choose which information to preserve and whether they use optimized compute kernels.

Weight-and-activation quantization compresses both weights and activations. FP8 W8A8 quantizes both to 8-bit FP8 (E4M3 format), enabling hardware-native FP8 tensor core operations on Ada Lovelace and Hopper GPUs.

3. Experimental Setup

3.1 Model

Mistral 7B Instruct v0.3 (`mistralai/Mistral-7B-Instruct-v0.3`), downloaded from Hugging Face and used as the single FP16 base checkpoint for all quantization variants. At 7B parameters in FP16 the model fits on a single 24GB GPU, keeping the benchmark operationally simple.

All quantized variants (AWQ, GPTQ, FP8, NF4) were produced by running quantization scripts on-pod against this base checkpoint. AWQ and GPTQ used real benchmark prompts as calibration data; FP8 and NF4 require no calibration. No pre-quantized third-party models were used - this isolates the effect of each quantization method itself.

3.2 Quantization Methods

Five methods were selected to cover the major categories of production-viable PTQ approaches available through vLLM as of early 2026.

Method	Precision	Technique	Calibration	Kernel	Why Selected
Baseline FP16	16-bit	None	N/A	Standard	Reference point for quality and performance
BitsAndBytes NF4	4-bit weights	On-the-fly NF4 compression	None	Generic (no kernel optimization)	Simplest quantization path; no calibration, no checkpoint save, load-time only. Represents the lowest-effort option
AWQ INT4	4-bit weights	Activation-aware weight quantization	128 samples, in-domain	Fused AWQ GEMM	Leading one-shot PTQ method; analyzes activation magnitudes to protect salient weight channels
GPTQ INT4 + Marlin	4-bit weights	Hessian-based layer-wise quantization	128 samples, seq_len=2048, in-domain	Marlin (Ampere-optimized CUDA)	Second-order weight optimization with the fastest available INT4 inference kernel
FP8 W8A8	8-bit weights + activations	Dynamic per-token	Static per-channel	Hardware-native FP8	Only method that quantizes activations; hardware-native on

Method	Precision	Technique	Calibration	Kernel	Why Selected
		activation quantization	weight scales	tensor cores	Ada/Hopper - represents the direction GPU vendors are pushing

Kernel isolation variants - to separate the effect of the quantization algorithm from the inference kernel, two additional configurations were tested using the same checkpoints with swapped kernels:

Variant	Checkpoint	Kernel	Purpose
AWQ INT4 (Marlin)	AWQ INT4 checkpoint	Marlin (Ampere-optimized CUDA)	Test whether AWQ weights achieve GPTQ-level throughput when given the same kernel
GPTQ INT4 (Generic)	GPTQ INT4 checkpoint	Generic GPTQ GEMM	Test GPTQ throughput without Marlin's kernel optimization

No re-quantization was needed - the same AWQ and GPTQ checkpoints were loaded with a different `vllm_quantization` flag.

Excluded methods: INT8 (SmoothQuant, LLM.int8()) was excluded - 2x the memory of INT4 with less mature vLLM kernel support than FP8. GGUF/llama.cpp was excluded as this study focuses on vLLM-based serving.

AWQ and GPTQ use in-domain benchmark prompts for calibration to better match activation statistics. FP8 and NF4 require no calibration.

3.3 Hardware and Infrastructure

All compute runs on **RunPod GPU pods** with interactive SSH access. Each method was primarily benchmarked on the GPU architecture best suited to its kernel requirements:

Method	GPU	Architecture	VRAM	RunPod $/hr	Rationale
Baseline FP16	A100 SXM	Ampere	80 GB	$1.49	Same-hardware reference for AWQ/GPTQ comparison
BitsAndBytes NF4	RTX A5000	Ampere	24 GB	$0.27	No special kernel needed; cheapest viable Ampere GPU

Method	GPU	Architecture	VRAM	RunPod $/hr	Rationale
AWQ INT4	A100 SXM	Ampere	80 GB	$1.49	Fused AWQ kernels optimized for Ampere
GPTQ INT4 + Marlin	A100 SXM	Ampere	80 GB	$1.49	Marlin kernel specifically optimized for Ampere
FP8 W8A8	L4	Ada Lovelace	24 GB	$0.39	FP8 tensor cores hardware-native on Ada; not supported on Ampere

Additionally, to enable direct cross-hardware comparisons, a subset of methods was re-run on alternate GPUs:

- Baseline FP16, AWQ INT4, GPTQ INT4, and BitsAndBytes NF4 were also benchmarked on L4
- BitsAndBytes NF4 was also benchmarked on A100 SXM
- This enables same-hardware comparisons on L4 and A100, along with cross-GPU analysis for NF4

Software stack: vLLM 0.6.6.post1, PyTorch 2.5.1 + CUDA 12.1, autoawq 0.2.7, auto-gptq (source build), llmcompressor >= 0.4.0, bitsandbytes >= 0.44.0.

3.4 Benchmark Prompt Suite

65 prompts across five categories, each designed to stress different aspects of LLM serving:

Category	Count	Input Length	Output Length	Stress Type
Travel & Dining	18	130-155 tokens	350-450 tokens	Generation-heavy; multi-option ranking
Credit Card Recommendation	18	140-145 tokens	280-320 tokens	Comparative reasoning; tradeoff explanation
Merchant Offer Matching	6	Long (retrieved context)	Short (top-3 list)	Prefill-heavy; stresses TTFT and KV-cache
Multi-Turn Conversation	5	Accumulating turns	Variable	Stateful context management
Structured JSON Output	18	Variable	JSON schema	Format compliance; exposes quantization errors

Prompts were generated from 30 hand-written seed prompts with context-override augmentation (e.g., varying city, budget, cuisine constraints). All prompts include system instructions aligned to Chase Sapphire concierge and recommendation personas.

3.5 Quality Evaluation

Quality is measured using **deterministic rule-based checks** (field presence, JSON schema validation, or hybrid). A request passes if >= 80% of its checks pass. The aggregate quality pass rate is the fraction of passing requests across all prompts.

This study focuses on serving performance — quality checks are a sanity gate confirming outputs are structurally valid, not a deep evaluation of semantic quality.

3.6 Metrics

Metric	Definition	What It Measures
TTFT (mean, p50, p95)	Time from request submission to first generated token	User-perceived responsiveness; grows with input length
TPS Total	Total output tokens generated per second across all concurrent requests	System-level throughput capacity
TPS Mean	Average per-request throughput (tokens / request wall time)	Per-user generation speed
Quality Pass Rate	Fraction of requests where >= 80% of rule-based checks pass	Output correctness regression from quantization
Cost per 1M Tokens	RunPod hourly rate / (TPS total * 3600) * 1,000,000	Inference cost efficiency accounting for GPU price and throughput

Concurrency levels tested: 1, 4, 8, 16. Concurrency 1 measures single-request latency; higher levels measure throughput scaling under load.

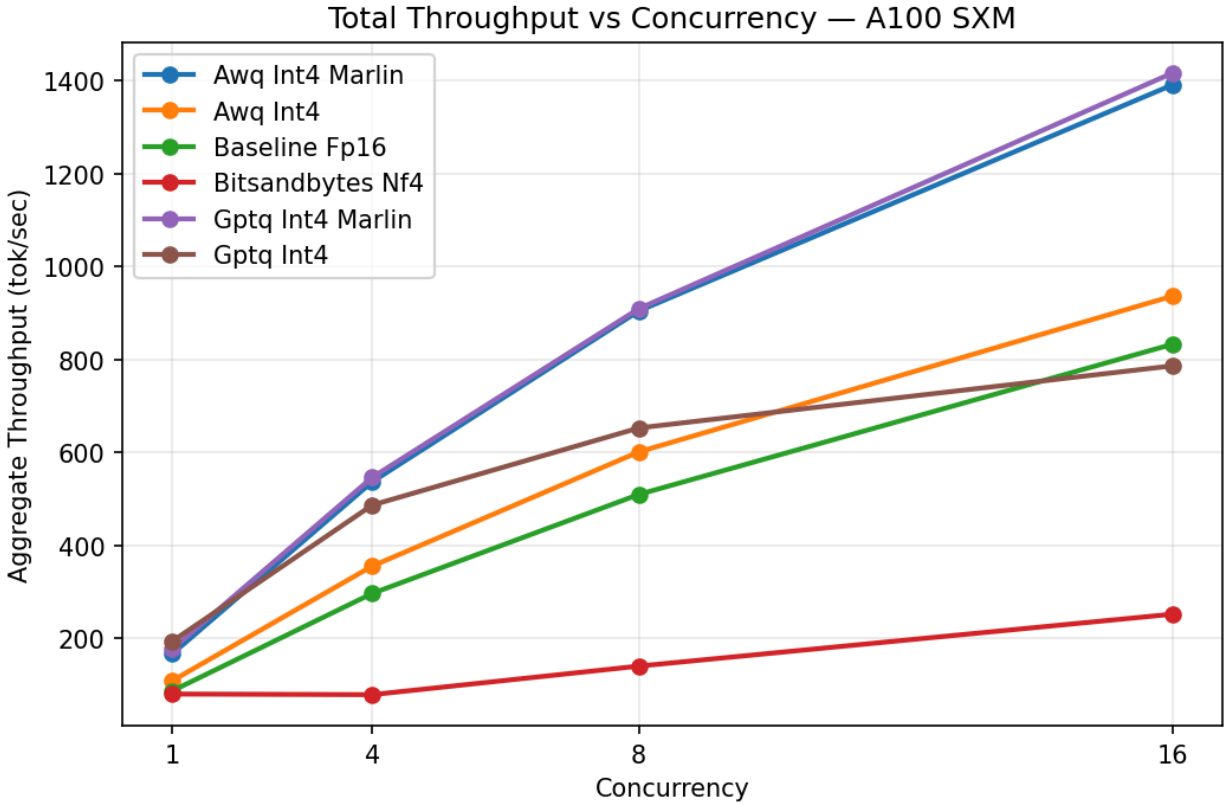
Temperature: Set to 0.0 (greedy decoding) for reproducibility across runs.

4. Results

4.1 Throughput (TPS Total)

A100 SXM — Methods compared on identical hardware:

Method	c=1	c=4	c=8	c=16	Scaling (c16/c1)
GPTQ INT4 + Marlin	178.2	572.1	958.2	1,452.2	8.2x
AWQ INT4 + Marlin	167.1	537.1	904.6	1,390.6	8.3x
AWQ INT4	109.2	355.8	601.5	936.4	8.6x
Baseline FP16	87.4	297.0	510.0	832.8	9.5x
GPTQ INT4 (Generic)	194.0	487.0	652.9	786.5	4.1x
BitsAndBytes NF4	80.7	79.4	140.8	252.4	3.1x



Both Marlin-backed methods converge at c=16 (~1,400 TPS) regardless of whether the weights

are AWQ or GPTQ \- suggesting that, in this benchmark, kernel choice appears to dominate high-concurrency throughput. GPTQ without Marlin starts fastest at c=1 (194 TPS) but scales poorly (4.1x), falling below AWQ's native kernel at c=16.

L4 - Methods compared on identical hardware:

Method	**c=1**	**c=4**	**c=8**	**c=16**	**Scaling (c16/c1)**
GPTQ INT4 Marlin	56.7	171.3	279.8	432.2	7.6x
AWQ INT4	50.3	156.4	255.3	381.8	7.6x
FP8 W8A8	28.1	90.5	155.5	248.9	8.9x
BitsAndBytes NF4	44.3	27.7	49.9	87.5	2.0x
Baseline FP16	17.4	56.8	98.5	164.4	9.4x

4.2 Latency (TTFT)

A100 SXM - TTFT at concurrency=1 (single-request latency):

Method	**Mean**	**p50**	**p95**
Baseline FP16	0.036s	0.031s	0.122s
GPTQ INT4 + Marlin	0.039s	0.028s	0.142s
AWQ INT4 + Marlin	0.041s	0.030s	0.150s
GPTQ INT4	0.047s	0.041s	0.131s
AWQ INT4	0.071s	0.064s	0.155s
BitsAndBytes NF4	0.081s	0.069s	0.180s

L4 - TTFT at concurrency=1:

Method	**Mean**	**p50**	**p95**
GPTQ INT4 Marlin	0.115s	0.076s	0.506s
FP8 W8A8	0.119s	0.082s	0.497s

Method	Mean	p50	p95
Baseline FP16	0.141s	0.118s	0.473s
AWQ INT4	0.223s	0.198s	0.570s
BitsAndBytes NF4	0.230s	0.196s	0.610s

TTFT scales roughly linearly with concurrency across all methods and GPUs. At concurrency=16, mean TTFT on L4 is around 1.5 seconds or higher for all methods.

4.3 Cost Efficiency

Cost per 1M tokens at concurrency=16 (highest throughput):

Rank	Method	GPU	$/1M Tokens
1	GPTQ INT4 + Marlin	L4 ($0.39/hr)	$0.2506
2	AWQ INT4	L4 ($0.39/hr)	$0.2837
3	GPTQ INT4 + Marlin	A100 SXM ($1.49/hr)	$0.2850
4	AWQ INT4 + Marlin	A100 SXM ($1.49/hr)	$0.2976
5	FP8 W8A8	L4 ($0.39/hr)	$0.4353
6	AWQ INT4	A100 SXM ($1.49/hr)	$0.4420
7	BitsAndBytes NF4	RTX A5000 ($0.27/hr)	$0.4772
8	Baseline FP16	A100 SXM ($1.49/hr)	$0.4970
9	GPTQ INT4 (Generic)	A100 SXM ($1.49/hr)	$0.5263
10	Baseline FP16	L4 ($0.39/hr)	$0.6588
11	BitsAndBytes NF4	L4 ($0.39/hr)	$1.2384
12	BitsAndBytes NF4	A100 SXM ($1.49/hr)	$1.6396

4.4 Quality

Quality pass rate at concurrency=1:

Method	GPU	Overall	Credit Card	Merchant Offers	Multi-Turn	Structured JSON	Travel & Dining
FP8 W8A8	L4	61.5%	65.4%	91.0%	74.5%	60.8%	85.7%
AWQ INT4	A100/L4	55.4%	66.7%	95.8%	68.3%	66.7%	82.2%
Baseline FP16	A100/L4	53.8%	67.6%	91.3%	73.7%	50.0%	83.3%
AWQ INT4 + Marlin	A100	52.3%	67.4%	95.8%	66.2%	66.7%	81.6%
GPTQ INT4 + Marlin	A100	52.3%	65.4%	87.0%	72.0%	60.0%	81.7%
GPTQ INT4	A100	53.8%	65.9%	91.7%	74.0%	49.2%	82.5%
GPTQ INT4 + Marlin	L4	50.8%	60.8%	91.0%	72.0%	54.5%	80.3%
BitsAndBytes NF4	A100	49.2%	63.0%	84.2%	75.8%	54.5%	83.3%
BitsAndBytes NF4	L4/A5000	43.1%	63.0%	84.0%	72.0%	49.2%	80.7%

Quality is largely consistent across GPUs and kernels. AWQ scores 55.4% on both A100 and L4, baseline FP16 scores 53.8% on both, and the kernel-swap variants (AWQ+Marlin at 52.3%, GPTQ Generic at 53.8%) are within the same narrow band. Changing the inference kernel does not affect output quality - as expected, since the kernel only changes how matmuls execute, not the weight values.

4.5 Concurrency Scaling

Scaling ratio (TPS total at c=16 / TPS total at c=1):

Method	A100	L4	RTX A5000
Baseline FP16	9.5x	9.4x	—
FP8 W8A8	—	8.9x	—
AWQ INT4	8.6x	7.6x	—
AWQ INT4 + Marlin	8.3x	—	—
GPTQ INT4 + Marlin	8.2x	7.6x	—
GPTQ INT4 (Generic)	**4.1x**	—	—
BitsAndBytes NF4	**3.1x**	**2.0x**	**2.4x**

All methods with optimized kernels (Marlin or fused AWQ) achieve 7.5-9.5x throughput scaling from concurrency 1 to 16. GPTQ without Marlin scales only 4.1x - better than BitsAndBytes NF4 (2-3x) but far below its Marlin-backed counterpart (8.0x), demonstrating that kernel optimization is critical for concurrency scaling.

4.6 Kernel Isolation Study

To determine whether the throughput advantage of GPTQ+Marlin over AWQ was due to the quantization algorithm or the inference kernel, we ran two additional experiments on A100 SXM using the same checkpoints with swapped kernels:

	AWQ Kernel	Marlin Kernel
AWQ weights	936 TPS (native)	1,391 TPS (+48%)
GPTQ weights	786 TPS (-16%)	1,416 TPS (native)

All values at concurrency=16 on A100 SXM.

Key observations:

- **AWQ + Marlin (1,391) ≈ GPTQ + Marlin (1,416):** Within 2% - the quantization algorithm is irrelevant when both use Marlin. The original gap appears to be driven primarily by kernel choice.
- **GPTQ without Marlin (786) < AWQ with native kernel (936):** GPTQ's generic kernel scales poorly (4.1x vs AWQ's 8.6x), producing lower throughput at c=16 despite starting faster at c=1.

- **GPTQ generic is fastest at c=1 (194 TPS)** but saturates under concurrency, suggesting its dequantization path is not optimized for batched inference.
- **Quality is unchanged by kernel swap** - all four configurations score 49-55%, confirming the kernel affects only computation speed, not output.

5. Discussion and Takeaways

5.1 The Inference Kernel Drives Throughput, Not the Quantization Algorithm

The kernel isolation study (Section 4.6) provides direct evidence: **the Marlin CUDA kernel accounts for nearly all of the throughput advantage previously attributed to GPTQ.** When both AWQ and GPTQ weights are served through Marlin, they produce nearly identical throughput (1,391 vs 1,416 TPS at c=16 on A100) - a 2% difference, well within noise. When GPTQ is served through its generic kernel instead of Marlin, throughput drops to 786 TPS, below AWQ's native-kernel result of 936.

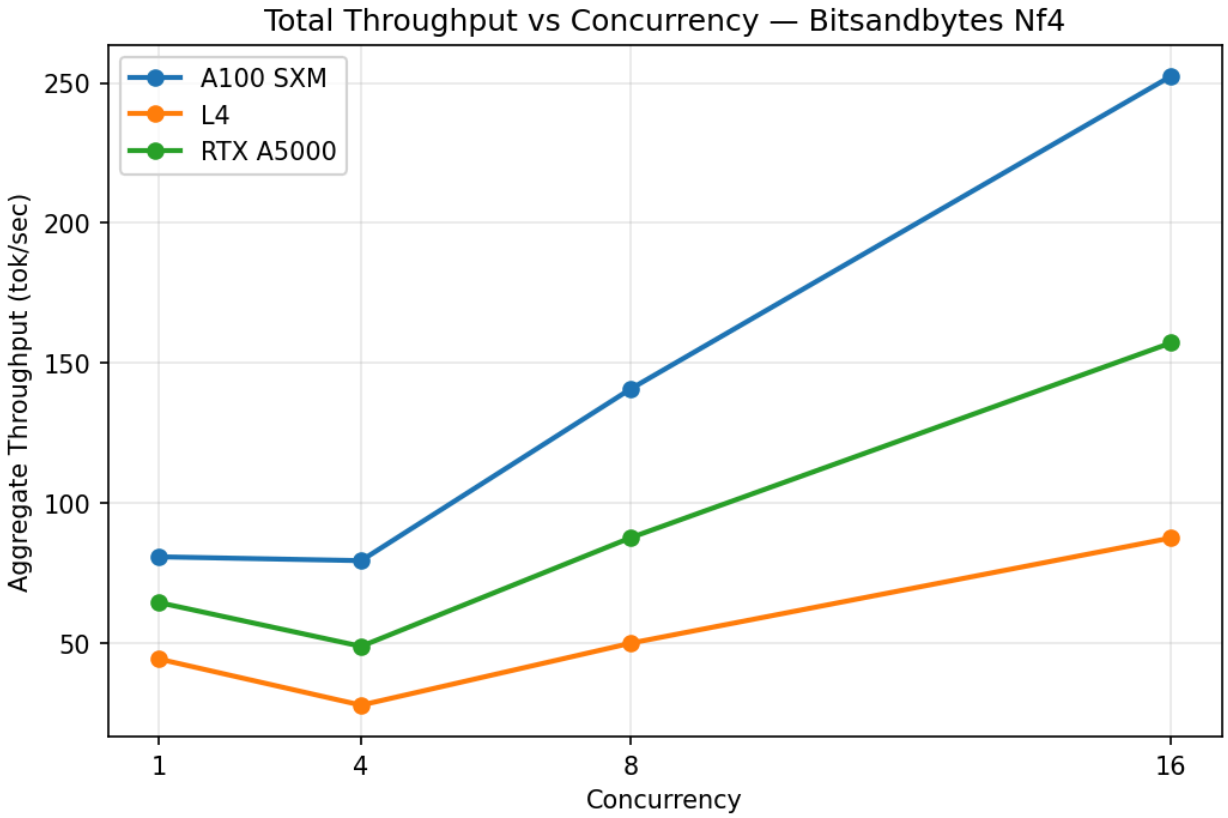
This means the original comparison (GPTQ+Marlin at 1,452 vs AWQ at 936) was confounded - it compared two variables simultaneously: the quantization algorithm and the inference kernel. The factorial experiment cleanly separates them and shows the kernel is the dominant factor at production-relevant concurrency levels.

For production deployments, the recommendation is to **use the Marlin kernel with whichever INT4 method is preferred** (AWQ or GPTQ). The choice of quantization algorithm is secondary to kernel selection for throughput.

5.2 BitsAndBytes NF4 Does Not Scale Under Concurrent Load

BitsAndBytes NF4 achieves competitive single-request throughput - 80.7 TPS on A100 at c=1, close to baseline FP16's 87.4. But it collapses under concurrency:

- **A100:** TPS mean drops from 80.5 (c=1) to 20.8 (c=4) — a 75% drop at just 4 concurrent requests
- **L4:** TPS mean drops from 44.1 (c=1) to 7.5 (c=4) — an 83% drop
- **RTX A5000:** TPS mean drops from 64.2 (c=1) to 12.9 (c=4) — an 80% drop



The scaling collapse is consistent across A100, L4, and RTX A5000 — ruling out hardware-specific causes.

This pattern is consistent across all three GPUs, ruling out hardware-specific issues. The likely cause is architectural: BitsAndBytes performs on-the-fly dequantization during each forward pass. At concurrency=1, the dequantization overhead is amortized over a single request. At higher concurrency, the repeated dequantization across concurrent batches likely saturates compute, negating the memory savings that quantization was supposed to provide. (Note: this explanation is consistent with the observed data but has not been confirmed with kernel profiling or ablation studies.)

By contrast, AWQ and GPTQ use pre-computed quantized representations with fused kernels that perform dequantization and computation in a single pass. This is why they maintain 7.5-8.6x scaling while BitsAndBytes achieves only 2-3x.

BitsAndBytes NF4 is a reasonable choice for prototyping, development, or single-user inference where simplicity matters. It should not be used for any workload expecting concurrent requests. At concurrency=16, BitsAndBytes on A100 ($1.64/1M tokens) costs **5.8x more** than GPTQ on A100 ($0.28/1M tokens) and **6.5x more** than GPTQ on L4 ($0.25/1M tokens).

5.3 Quantized Models on Cheap GPUs Can Beat Unquantized Models on Expensive GPUs

GPTQ on L4 achieves ~50% lower cost per token than FP16 on A100. The measured cost-per-token results favor L4-backed INT4 configurations over the A100 FP16 baseline. See Section 4.3 for the full cost ranking.

5.4 FP8 Produced the Highest Structural Pass Rate, with Throughput Below INT4 but Above FP16 on L4

FP8 W8A8 achieves the highest quality pass rate in the study at 61.5% — meaningfully above baseline FP16 (53.8%). This is counterintuitive: a quantized model outperforming the unquantized baseline on quality.

A possible explanation is that FP8's 8-bit precision retains more weight information than INT4 methods, and the FP8 E4M3 format has a wider dynamic range, reducing the probability of catastrophic rounding errors on outlier weight values.

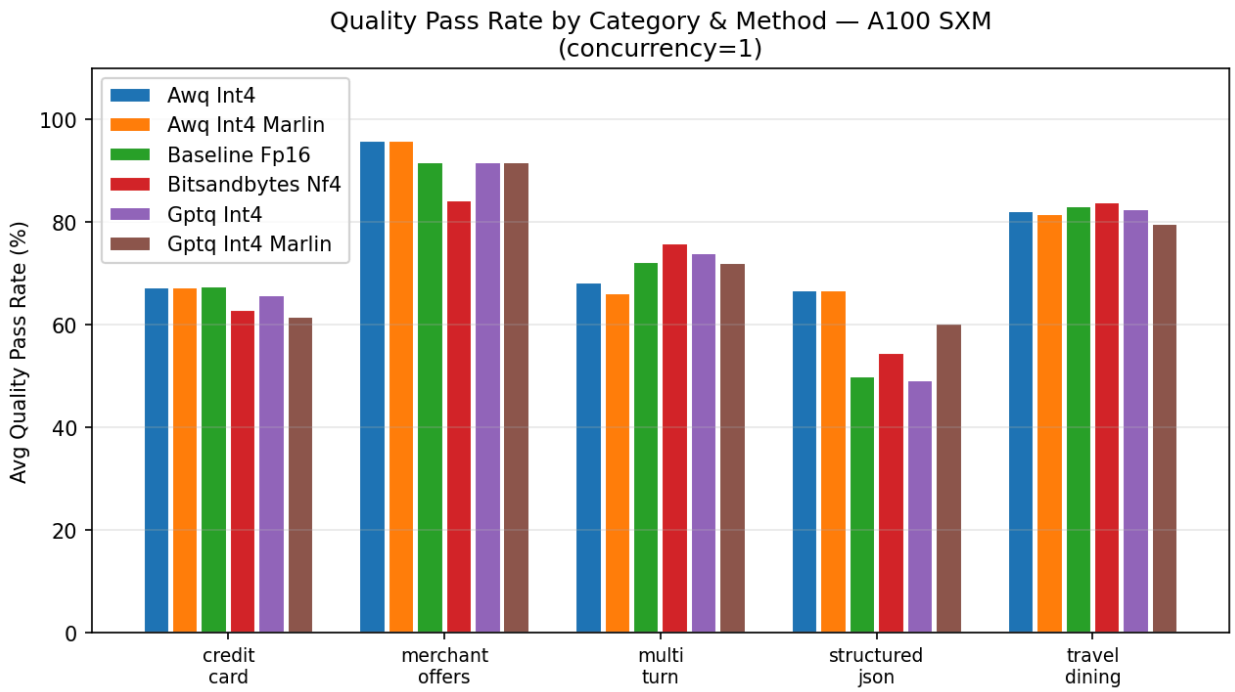
On throughput, FP8 on L4 (248.9 TPS at c=16) sits between the INT4 methods (382-432 TPS) and baseline FP16 (164.4 TPS). Its 8.9x concurrency scaling is healthy, confirming that hardware-native FP8 tensor cores work as intended on Ada Lovelace.

FP8 may be worth considering when output correctness is a priority and the workload runs on Ada or Hopper hardware, though absolute quality scores remained modest across all methods in this benchmark.

5.5 Quality Is More Sensitive to Category Than to Quantization Method

Across all methods, the quality pattern by category is remarkably consistent:

- **Merchant offers** score highest (84-96%) - these prompts require extracting and listing information from provided context, a task where quantization has minimal impact
- **Travel & dining** score high (80-86%) - structured recommendation lists with clear constraints
- **Multi-turn** scores moderate (68-76%) - stateful conversation tracking is moderately robust
- **Credit card** scores moderate (61-68%) - comparative reasoning across multiple options
- **Structured JSON** scores lowest (49-67%) - precise format compliance is most sensitive to quantization



Within each category, methods cluster tightly. Across categories, scores vary by 25-40 percentage points - task difficulty dominates quantization effects.

The spread between the best and worst quantization method within a single category (typically 5-15 percentage points) is smaller than the spread between the best and worst category for a single method (typically 25-40 percentage points). This suggests that **prompt design and task difficulty have a larger effect on output quality than quantization method choice**, at least within the INT4-FP8-FP16 precision range tested here.

5.6 FP16 Is Impractical on 24GB GPUs

Baseline FP16 on L4 is the worst configuration in every metric except quality:

- Lowest throughput: 17.4 TPS at c=1, 164.4 at c=16
- Highest cost: $6.21/1M tokens at c=1
- Slowest TTFT: 0.141s mean at c=1

Every quantization method dramatically improves on FP16 performance on L4. GPTQ delivers 3.2x the single-request throughput and AWQ delivers 2.9x. Even BitsAndBytes NF4, despite its scaling problems, provides 2.5x better single-request TPS.

Running a 7B model unquantized on a 24GB GPU is technically possible but economically wasteful. On 24GB GPUs, quantization was necessary to achieve competitive throughput and cost in these runs.

5.7 Limitations and Caveats

This benchmark does not simulate production workloads. All requests use a fixed prompt suite with deterministic decoding (temperature=0.0). Real production traffic has variable request lengths, bursty arrival patterns, and stochastic decoding. The concurrency levels tested (1-16) represent static batching, not the continuous batching that vLLM uses in online serving mode. Results should be interpreted as controlled comparisons between methods, not as predictions of production performance.

The quality evaluation is structural, not semantic. Rule-based checks measure whether required fields are present and JSON is valid, not whether recommendations are actually good. A response that mentions "restaurant" and "reservation" passes the check regardless of whether the restaurant exists or the recommendation makes sense. Semantic quality differences between methods may exist but are not captured here.

TPS mean may not reflect individual user experience under load. The `tps_mean` metric measures per-request throughput as tokens divided by total batch wall time. Under high concurrency, multiple requests share the same wall time window, so `tps_mean` decreases even though the system is producing more total tokens. `tps_total` is the more meaningful metric for system capacity.

6. Recommendations

These recommendations should be validated with representative production traffic and SLA requirements.

- **Throughput:** INT4 + Marlin kernel (AWQ or GPTQ — both perform equivalently with Marlin, see Section 4.6). AWQ may be preferred for simpler quantization; GPTQ for marginally better single-request latency.
- **Quality:** FP8 W8A8 on Ada/Hopper GPUs, noting that absolute scores remained modest across all methods (best: 61.5%).
- **Cost:** INT4 methods (GPTQ or AWQ) + Marlin on L4 — cheapest cost per token in this benchmark.
- **Prototyping:** BitsAndBytes NF4 for single-user inference only — unsuitable under concurrency.
- **Kernel matters more than algorithm:** The kernel isolation study shows that switching from a generic kernel to Marlin yields up to 80% throughput gain at c=16, while switching between AWQ and GPTQ weights on the same kernel yields <2% difference.
- **Avoid:** Unquantized FP16 on 24GB-class GPUs; GPTQ with its generic kernel under concurrent load (4.1x scaling vs 8.0x with Marlin).

Appendix

A. Full Summary Data

A100 SXM — All Methods

Method	Concurrency	TTFT Mean (s)	TTFT p50 (s)	TTFT p95 (s)	TPS Mean	TPS Total	Quality %	$/1M Tokens
baseline_fp16	1	0.0362	0.0305	0.1224	87.24	87.37	53.85%	$4.7374
baseline_fp16	4	0.1084	0.0774	0.4070	77.54	297.04	55.38%	$1.3934
baseline_fp16	8	0.2080	0.1374	0.5174	70.16	509.96	55.38%	$0.8116
baseline_fp16	16	0.3657	0.2623	0.8810	62.89	832.80	55.38%	$0.4970
awq_int4	1	0.0706	0.0644	0.1552	108.92	109.22	55.38%	$3.7894
awq_int4	4	0.1466	0.1127	0.4415	92.01	355.80	53.85%	$1.1633
awq_int4	8	0.2410	0.1717	0.5493	82.00	601.51	52.31%	$0.6881
awq_int4	16	0.4027	0.2956	0.9519	68.77	936.35	52.31%	$0.4420
gptq_int4_marlin	1	0.0388	0.0283	0.1420	177.61	178.17	52.31%	$2.3230
gptq_int4_marlin	4	0.1282	0.0855	0.5058	148.56	572.12	53.85%	$0.7234
gptq_int4_marlin	8	0.2540	0.1649	0.6329	131.51	958.20	53.85%	$0.4319
gptq_int4_marlin	16	0.4526	0.3244	1.0888	108.17	1452.19	53.85%	$0.2850
bitsandbytes_nf4	1	0.0806	0.0688	0.1800	80.55	80.74	49.23%	$5.1264
bitsandbytes_nf4	4	0.1662	0.1270	0.5153	20.80	79.36	47.69%	$5.2152
bitsandbytes_nf4	8	0.2812	0.2001	0.6495	18.81	140.80	46.15%	$2.9396
bitsandbytes_nf4	16	0.4912	0.3478	1.1144	17.49	252.44	46.15%	$1.6396
awq_int4_marlin	1	0.0406	0.0297	0.1495	166.56	167.05	52.31%	$2.4776
awq_int4_marlin	4	0.1353	0.0904	0.5328	139.39	537.13	53.85%	$0.7706

Method	Concurrency	TTFT Mean (s)	TTFT p50 (s)	TTFT p95 (s)	TPS Mean	TPS Total	Quality %	$/1M Tokens
awq_int4_marlin	8	0.2669	0.1747	0.6647	123.76	904.58	55.38%	$0.4575
awq_int4_marlin	16	0.4747	0.3407	1.1403	102.77	1390.63	55.38%	$0.2976
gptq_int4	1	0.0473	0.0411	0.1311	193.48	193.98	53.85%	$2.1336
gptq_int4	4	0.1167	0.0826	0.4063	125.92	487.02	52.31%	$0.8498
gptq_int4	8	0.2143	0.1495	0.5152	87.15	652.93	50.77%	$0.6339
gptq_int4	16	0.3684	0.2666	0.8818	54.44	786.46	49.23%	$0.5263

L4 — All Methods

Method	Concurrency	TTFT Mean (s)	TTFT p50 (s)	TTFT p95 (s)	TPS Mean	TPS Total	Quality %	$/1M Tokens
baseline_fp16	1	0.1406	0.1175	0.4730	17.42	17.44	53.85%	$6.2103
baseline_fp16	4	0.4406	0.2923	1.7115	14.80	56.79	55.38%	$1.9077
baseline_fp16	8	0.8591	0.5484	2.1083	13.51	98.47	56.92%	$1.1002
baseline_fp16	16	1.4980	1.0724	3.6563	12.44	164.43	53.85%	$0.6588
awq_int4	1	0.2229	0.1976	0.5702	50.12	50.34	55.38%	$2.1518
awq_int4	4	0.5476	0.3969	1.9043	40.44	156.35	55.38%	$0.6929
awq_int4	8	0.9810	0.6739	2.3533	34.77	255.30	55.38%	$0.4243
awq_int4	16	1.6782	1.2094	4.0655	27.88	381.84	52.31%	$0.2837
gptq_int4_marlin	1	0.1145	0.0762	0.5059	56.53	56.68	50.77%	$1.9114
gptq_int4_marlin	4	0.4662	0.3229	1.8686	44.71	171.27	52.31%	$0.6325
gptq_int4_marlin	8	0.9286	0.6386	2.3460	38.62	279.76	52.31%	$0.3872
gptq_int4_marlin	16	1.6639	1.2020	4.0076	32.31	432.22	53.85%	$0.2506

Method	Concurrency	TTFT Mean (s)	TTFT p50 (s)	TTFT p95 (s)	TPS Mean	TPS Total	Quality %	$/1M Tokens
fp8_w8a8	1	0.1189	0.0823	0.4971	28.03	28.09	61.54%	$3.8566
fp8_w8a8	4	0.4580	0.3008	1.8469	23.56	90.45	55.38%	$1.1978
fp8_w8a8	8	0.9103	0.5697	2.3413	21.32	155.50	56.92%	$0.6967
fp8_w8a8	16	1.5959	1.1447	3.9244	18.64	248.88	56.92%	$0.4353
bitsandbytes_nf4	1	0.2296	0.1960	0.6101	44.07	44.26	43.08%	$2.4478
bitsandbytes_nf4	4	0.5741	0.3963	2.0584	7.47	27.74	41.54%	$3.9055
bitsandbytes_nf4	8	1.0564	0.7022	2.5500	6.86	49.92	49.23%	$2.1702
bitsandbytes_nf4	16	1.8991	1.3024	4.4088	6.19	87.48	46.15%	$1.2384

RTX A5000 — BitsAndBytes NF4

Method	Concurrency	TTFT Mean (s)	TTFT p50 (s)	TTFT p95 (s)	TPS Mean	TPS Total	Quality %	$/1M Tokens
bitsandbytes_nf4	1	0.1534	0.1332	0.4303	64.17	64.43	43.08%	$1.1641
bitsandbytes_nf4	4	0.3873	0.2578	1.3545	12.93	48.75	47.69%	$1.5386
bitsandbytes_nf4	8	0.6997	0.4607	1.6699	11.87	87.62	46.15%	$0.8560
bitsandbytes_nf4	16	1.2494	0.8635	2.8583	10.94	157.17	46.15%	$0.4772

B. RunPod GPU Hourly Rates

GPU	Architecture	VRAM	RunPod $/hr
RTX A5000	Ampere (GA102)	24 GB	$0.27
L4	Ada Lovelace	24 GB	$0.39
A100 SXM	Ampere (GA100)	80 GB	$1.49

Cost per 1M tokens is calculated as: `(hourly_rate / (tps_total * 3600)) * 1,000,000`.

C. Quantization Configurations

AWQ INT4:

- Library: autoawq 0.2.7
- Group size: 128, Weight bits: 4, Version: GEMM
- Calibration: 128 in-domain samples
- Quantization time: ~10 minutes on A100

GPTQ INT4 + Marlin:

- Library: auto-gptq (source build)
- Bits: 4, Group size: 128, Symmetric: true, Desc act: false (required for Marlin)
- Calibration: 128 in-domain samples, sequence length: 2048
- Damping percent: 0.01
- Quantization time: ~20 minutes on A100

AWQ INT4 + Marlin (kernel isolation variant):

- Same AWQ INT4 checkpoint as above (no re-quantization)
- vLLM quantization flag: `awq_marlin` (routes through Marlin kernel instead of fused AWQ kernel)

GPTQ INT4 Generic (kernel isolation variant):

- Same GPTQ INT4 checkpoint as above (no re-quantization)
- vLLM quantization flag: `gptq` (routes through generic GPTQ kernel instead of Marlin)
- Note: checkpoint was quantized with `desc_act=False` (Marlin-compatible). A `desc_act=True` checkpoint could yield marginally better quality with the generic kernel but was not tested.

FP8 W8A8:

- Library: llmcompressor >= 0.4.0
- Scheme: FP8_DYNAMIC (E4M3 format)
- Targets: all Linear layers (excluding lm_head)
- Weights: static per-channel quantization (no calibration data needed)
- Activations: dynamic per-token quantization at inference

BitsAndBytes NF4:

- Library: bitsandbytes >= 0.44.0
- Format: Normal Float 4-bit (NF4)
- Compute dtype: float16
- No calibration, no checkpoint — quantization happens at model load time

D. Software Versions

Component	**Version**
vLLM	0.6.6.post1
PyTorch	2.5.1 + CUDA 12.1
autoawq	0.2.7
auto-gptq	Source build
llmcompressor	>= 0.4.0
compressed-tensors	0.8.1
bitsandbytes	>= 0.44.0
transformers	4.45-4.46
Python	3.10+

E. Benchmark Configuration

See Section 3.6 for metrics definitions. Key parameters: max tokens 512, temperature 0.0, context length 8192, warmup 5, concurrency [1, 4, 8, 16], 65 requests per level.